⚡ Schematic2DSN: Technical Architecture & Solution

Automated Circuit Graph Reconstruction & Cadence OrCAD Compiler

Developed by **Jatin Makhija** (@JatinMakhija-10) • Repository: github.com/JatinMakhija-10/Schematic2DSN • Python 3.11+ • 163 Tests Passing

> **The Core Engineering Thesis:** We do not simply draw a picture or produce a flat raster replica of a schematic; we reconstruct the underlying circuit topology graph and compile it into a native, fully editable, journal-verified Cadence OrCAD Capture design (.DSN) placed on the 100-mil electrical snap grid.

1. The Problem: Legacy Schematic Lock-In

Millions of mission-critical electrical designs exist only as static PDFs, scanned blueprints, or legacy image archives. Engineers needing to modify, simulate, or fabricate these circuits face hours or days of error-prone manual labor in EDA software: manually redrawing symbols, reconnecting every pin, and re-typing labels.

Existing optical approaches fail because standard OCR/computer vision outputs dumb pixels. Circuit simulation and PCB layout require **strict topological netlists**, where pin numbers govern physical connectivity and wires snap to a discrete 100-mil grid.

2. End-to-End Compiler Architecture

Schematic2DSN operates as a deterministic 11-stage compiler pipeline. Every stage reads and writes verified Pydantic JSON contracts, allowing pure-function modularity and rigorous unit testing:

Stage	Module	Input & Output Contract	Key Engineering Function
S1 Ingest	`vision.ingest`	PDF/PNG → `page.png`, `page.meta.json`	Dual-pathway vector/raster routing, skew correction, DPI checks
S2 Text	`vision.ocr`	`page.png` → `texts.json`	0°/90° RapidOCR sweeps + electronic regex normalization
S3 Symbols	`vision.symbols_cv`	`page.png` → `symbols.json`	Text masking + geometric heuristics (13 classes) + YOLOv8
S4 Wires	`vision.wires`	`page.png` → `wires.json`	Symbol masking + medial axis skeletonization + Hough transform
S5 Junctions	`vision.junctions`	`page.png` → `junctions.json`	Morphological disk opening (circularity > 0.7) for solder dots
S6 Pins	`vision.pins`	`symbols.json` → `pins.json`	Rigid EDA library offsets snapped to wire endpoints (≤ 6px)
S7 Graph	`circuit.graph`	Wires/Junctions → `nets_raw.json`	Union-Find (DSU) engine + global net alias label propagation
S8 Associate	`circuit.associate`	Texts/Symbols → `components_raw.json`	Bipartite Hungarian optimization for RefDes and values
S9 Validate	`circuit.validate`	Raw Graph → `design.json` (IR)	Electrical Rules Check (ERC) + deterministic confidence score
S10 Export	`orcad.tcl_gen`	`design.json` → `design.tcl`	100-mil grid compile: Mode A (stubs) or Mode B (polylines)
S11 Roundtrip	`orcad.roundtrip`	`design.json` + Netlist → Proof	Canonical partition equivalence proof (frozenset of frozenset)

3. The Vision & Topological Extraction Engine

Stage 1: Dual-Pathway Ingest (Vector vs. Raster)

Vector Pathway: When processing vector PDFs, Schematic2DSN inspects internal drawing streams with PyMuPDF (`get_drawings()` and `get_text("dict")`). This extracts mathematically exact line endpoints and text characters directly from the file, bypassing raster distortion entirely.

Raster Pathway: For scans or photos, it renders at 600 DPI, runs Otsu adaptive thresholding, removes speckles (<6 px area), and deskews using dominant Hough angles. Rejection rules immediately flag multi-page PDFs, low resolution (<1200 px long side), or excessive line skew.

Stage 2: Text Extraction & Domain Regex Normalization

Schematic labels are often vertical. The OCR engine executes dual 0° and 90° sweeps with ONNX-accelerated RapidOCR, merging overlaps by confidence. Crucially, raw strings pass through domain-specific regex normalizers:

- **RefDes Repair:** Fixes optical character ambiguities (e.g., `RI` → `R1`, `QO` → `Q0`, `CI` → `C1`) according to standard electronics prefixes.
- **Engineering Unit Normalization:** Converts shorthand notation: `4k7` → `4.7k`, `100R` → `100ohm`, `.1uF` → `0.1uF`, `10K` → `10k`.
- **Net Alias Detection:** Maps standard power and control rails: `VCC`, `GND`, `+5V`, `3V3`, `RESET`, `SCL`, `SDA`.

Stage 3: Symbol Detection & Classification

Text bounding boxes are dilated and subtracted first, ensuring text characters are never mistaken for circuit shapes. A hybrid engine classifies 13 component types (Resistors, Capacitors, Inductors, Diodes, LEDs, BJTs, MOSFETs, Op-Amps, Connectors, Ground, VCC):

- **Geometric Heuristics (CV):** Resistors are identified by zig-zag turns (≥6) or IEC rectangles; Capacitors by parallel dielectric plates; Diodes/LEDs by triangles touching a cathode bar; Op-amps by triangular dual-input bodies.
- **YOLOv8 Detection:** Deep learning model trained on synthesized multi-rotation symbol datasets.
- **Orientation & Polarity:** Computes rotation (0°, 90°, 180°, 270°) and mirror status from physical asymmetry.

Stage 4 & 5: Wire Segmentation & Solder Dot Classification

Solder Dot Opening: Morphological opening with a circular structuring element larger than wire width erases thin lines and isolates solder dots. Connected components with circularity > 0.70 are registered as junctions.

Electronics Connectivity Rules:

- **4-Way Cross (+) without Dot:** Independent wire jumper (NOT connected).
- **4-Way Cross (+) with Solder Dot:** Bonded 4-way node (CONNECTED).
- **3-Way T-Junction (⊤):** Always an electrical connection (CONNECTED).

4. Graph Reconstruction, Association & Validation

Stage 6: EDA Library Contract & Pin Snapping

In electronic engineering, pin locations are strictly defined by library geometry, not by image artifacts. Schematic2DSN maintains an exact library contract in `orcad/symbols.json`. For a detected symbol at (x, y) with rotation θ, theoretical pin terminal coordinates are calculated and snapped to the nearest extracted wire endpoint within a calibrated 6-pixel threshold. Pins lacking attached wires are marked as `UNCONNECTED_PIN`.

Stage 7: Union-Find (DSU) Electrical Net Extraction

Wire segments, solder junctions, component pins, and net label markers form an undirected graph $G = (V, E)$. The system executes a Disjoint-Set Union (Union-Find) algorithm with path compression and union-by-rank:

- Every connected cluster of segments and pins collapses into a single **Electrical Net**.
- Identically named net aliases (e.g. `VCC`, `GND`, `RESET`) globally merge separated net sets.
- Nets are automatically assigned canonical deterministic identifiers (`NET_001`, `NET_002`) or preserve explicit label names.

Stage 8: Bipartite Matching via the Hungarian Algorithm

Associating free-floating text labels with nearby component symbols is modeled as a **Minimum-Weight Bipartite Matching Problem**. A global cost matrix is computed based on Euclidean distance, orientation alignment, and prefix compatibility ($R ightarrow ext{Resistor}$, $C ightarrow ext{Capacitor}$):

$$\textit{Cost} = \textit{Distance} + \textit{Prefix_Penalty} + \textit{Orientation_Penalty}$$

Solved using the Kuhn-Munkres (Hungarian) algorithm (`scipy.optimize.linear_sum_assignment`), ensuring each component receives its mathematically optimal RefDes and value without duplicates.

Stage 9: Electrical Rules Check (ERC) & Confidence Scoring

Before compilation, the intermediate representation (`design.json`) undergoes automated Electrical Rules Checking:

- Checks for dangling single-pin nets, unconnected component pins, and duplicate RefDes.
- Flags floating active inputs and missing passive values.

Deterministic Confidence Formula: Confidence is mathematically derived rather than guessed by an LLM:

```
Confidence = 0.35 × C_symbols + 0.25 × C_ocr + 0.25 × (Connected_Pins / Total_Pins) + 0.15 ×
(Valid_Nets / Total_Nets) − Σ Penalties
```

5. Cadence OrCAD Compiler & Automated Round-Trip Proof

Stage 10: Cadence OrCAD Capture Tcl Generation

Schematic2DSN translates the intermediate representation into clean, journal-verified Cadence Capture Tcl scripts:

- **Grid Snapping:** Image coordinates (Y-down) are converted to OrCAD mils (Y-up) and snapped to Cadence's strict **100-mil electrical snap grid**.
- **Mode A (Stub + Net Alias — Robust):** Places components, draws short orthogonal stubs from each pin, and attaches net aliases. OrCAD Capture natively bonds same-named aliases on the sheet. Highly robust and 100% editable.
- **Mode B (Routed Polylines — Geometric):** Renders full orthogonal routed wire polylines matching original schematic visual paths.

Stage 11: Canonical Netlist Round-Trip Proof

To mathematically prove that the generated design matches the input schematic without human bias:

1. The generated Tcl script is sourced in OrCAD Capture to build the native `.DSN` database.
2. Capture dumps its generated netlist.
3. Schematic2DSN parses both the input IR and the Capture netlist into **Canonical Partition Sets**:
   `S = frozenset( frozenset((refdes, pin_number), ...) )`
4. An automated set-equality check validates **100% topological equivalence**.

6. Interactive Streamlit Web Platform & Cloud Infrastructure

Schematic2DSN provides an engineer-friendly Streamlit web interface with real-time feedback:

- **Multi-Layer Vision Overlay:** Toggle between Raw Image, Text OCR, Wire Skeletons, Detected Symbols, and Solder Junctions.
- **Live IR Inspector & Tcl Downloader:** Immediate JSON tree inspector, ERC warning logs, and one-click `design.tcl` download.
- **Cloud Resilience:** Fully configured with `packages.txt` (providing `libgl1` and `libglib2.0-0`) to ensure headless Linux execution on Streamlit Community Cloud and Docker containers without graphics library failures.

7. Quality Assurance & Test Metrics

Metric	Value	Verification Method
Automated Test Suite	**163 Unit & Integration Tests Passing**	`pytest -v` in 6.2 seconds
Benchmark Circuits Suite	**12 Reference Ground-Truth Circuits**	Synthetic and golden reference schematics
Coordinate Grid Accuracy	**100% Snap on 100-mil Grid**	Verified in Cadence OrCAD Capture
Component Coverage	**13 Electrical Symbol Classes**	Passives, Discretes, ICs, Power Rails

8. Benchmark Circuits Suite (Ready-to-Test PDFs)

The repository contains 12 verified reference circuits with ground-truth netlists, available in `data/synthetic/` and `sample_schematics/`:

Circuit Benchmark	Components Included	Electrical Capabilities Tested
Voltage Divider	R1, R2, VCC, GND	Symmetric passive divider chain, vertical rail connection, baseline test
RC Low-Pass Filter	R1, C1, VCC, GND, OUT	Resistor-capacitor filtering, ground connection, output node extraction
LED Driver	R1, D1 (LED), VCC, GND	Diode polarity verification, current limiter passive, cathode-anode assignment
Inverting Op-Amp	U1, R1, R2, VCC, VEE	5-pin active op-amp symbol, input resistor, feedback loop, dual power rails
BJT Common Emitter	Q1 (NPN), R1, R2, RC, RE, VCC	Bipolar transistor terminals (Base, Collector, Emitter), bias network
BJT Astable Multivibrator	Q1, Q2, R1-R4, C1-C2, VCC	Cross-coupled symmetric oscillator, 4-way wire crossings, solder junctions
H-Bridge Motor Driver	M1-M4 (MOSFETs), J1, VCC	4-MOSFET power bridge, Gate-Drain-Source snapping, terminal connector
Differential Amplifier	U1, R1-R4, VCC, VEE	Balanced precision instrumentation stage, matched resistor network
RLC Bandpass Filter	R1, L1, C1, GND	Second-order passive filter including inductor coil symbol extraction
NE555 Timer Astable	U1 (8-pin IC), R1, R2, C1	Complex multi-pin IC: VCC, GND, TRIG, OUT, RESET, CONT, THRES, DISCH
LM2596 Buck Regulator	U1 (5-pin), L1, D1, C1, C2	Switching power supply topology, Schottky catch diode, power inductor
ATmega328P MCU System	U1 (28/32-pin), XTAL, C1-C3	High pin-count microcontroller, crystal oscillator tank, bypass passives

9. How to Run & Verify Live

1. Run the Entire Test Suite

`pytest -v` — Executes all 163 unit and integration tests across all compiler stages in <7 seconds.

2. Run CLI Pipeline on Any Benchmark PDF

```
python -m core.pipeline sample_schematics/voltage_divider.pdf out/ --generate-tcl
```

3. Launch Web Platform

`streamlit run app/streamlit_app.py` — Open http://localhost:8501 and upload any schematic PDF from `sample_schematics/`.

Schematic2DSN • Developed by Jatin Makhija (@JatinMakhija-10)
Source Code: https://github.com/JatinMakhija-10/Schematic2DSN